

实验六：基于 FUSE 的可读写内存文件系统（参考手册）

1. VFS 是什么

VFS 是 Virtual File System 的缩写，通常译作“虚拟文件系统”。这里的“虚拟”不是说它不真实，而是说它提供了一层统一抽象：无论底层是 ext4、tmpfs、proc，还是本实验中的 FUSE 文件系统，用户程序都可以用同一套系统调用访问它们。

例如，用户程序通常只会调用：

```
open / read / write / close / stat / unlink
```

它并不直接知道底层文件系统是什么。真正进入内核后，VFS 会负责：

1. 解析路径属于哪个挂载点。
2. 根据挂载表找到对应的文件系统。
3. 把“获取属性、打开、读取、写入、删除”等请求交给那个文件系统实现。
4. 把文件系统返回的结果再交还给用户程序。

所以 VFS 更像 Linux 内核中的“统一调度层”。它自己不负责决定 `/a.txt` 的内容是什么，也不负责把数据存到哪里；它负责把请求分发给正确的具体文件系统。

本实验写的 `memfs` 不是 VFS。`memfs` 是一个具体文件系统实现。它回答的是：

- 这个路径是否存在？
- 这个路径是目录还是普通文件？
- 目录下面有哪些名字？
- 文件内容从哪里读？
- 新写入的数据保存到哪里？

两者的区别可以概括为：

VFS: Linux 内核中的统一文件系统入口和分分层。

memfs: 本实验自己实现的一个具体文件系统，负责回答 VFS 转发来的请求。

执行下面命令时：

```
cat ~/memfs_mount/a.txt
```

调用链大致是：

```
cat
-> read/open 等系统调用
-> Linux VFS
-> 发现 ~/memfs_mount 是 FUSE 挂载点
-> FUSE 内核模块
-> 用户态 memfs 程序
-> memfs_read 等回调
-> files[] 内存数组
```

因此，参考手册中经常提到 VFS，是为了说明普通命令为什么会进入你写的回调函数。我们没有重写 `cat`，也没有改 Linux 的系统调用；我们只是实现了一个可以被 VFS 挂载和调用的文件系统。

2. 什么是挂载

2.1 为什么需要挂载

Linux 中所有路径都组织在同一棵目录树下，例如 `/`、`/home`、`/mnt`、`/proc`。但是这些路径背后不一定来自同一个文件系统。

例如：

- `/` 可能来自磁盘上的 ext4 文件系统。
- `/boot` 可能来自另一个磁盘分区。
- `/proc` 不是磁盘文件系统，而是内核暴露进程信息的伪文件系统。
- U 盘插入后，可能被挂载到 `/media/user/USB`。

挂载就是把一个文件系统接到目录树中的某个目录上。这个目录叫做挂载点。挂载完成后，访问这个目录下面的路径时，VFS 会把请求交给被挂载的文件系统处理。

可以把挂载理解成一条规则：

当访问某个目录及其子路径时，不再使用原来目录所在的文件系统，而是转交给挂载到这里的新文件系统。

所以挂载不是“复制文件”，也不是“创建快捷方式”。它改变的是：**某个路径下面的文件系统请求由谁来回答**。

2.2 真实文件系统的挂载

真实磁盘文件系统通常需要一个存储设备或镜像文件作为数据来源。例如，一个 ext4 分区可以这样挂载：

```
sudo mount -t ext4 /dev/sdb1 /mnt/disk
```

这条命令大致告诉内核：

`/dev/sdb1` 这个块设备里有一个 `ext4` 文件系统；
请把它接到 `/mnt/disk` 这个目录上；
以后访问 `/mnt/disk` 下面的路径时，交给 `ext4` 文件系统驱动处理。

挂载后，用户执行：

```
ls /mnt/disk
cat /mnt/disk/a.txt
```

`ls` 和 `cat` 仍然只是普通程序。它们调用系统调用后，VFS 根据挂载表发现 `/mnt/disk` 属于 ext4，于是把请求交给内核中的 ext4 驱动。ext4 驱动再去读取磁盘块、解析 inode、查目录项、取文件内容。

真实文件系统的挂载关系可以简化为：

```
用户程序
-> 系统调用
-> VFS
-> 挂载表判断路径属于 ext4
-> 内核 ext4 驱动
-> 读取或修改磁盘块
```

2.3 可以自己尝试的系统文件系统挂载例子

下面两个例子都不是 FUSE，而是 Linux 系统已经提供的文件系统。建议先亲自执行一下，再回来看本实验的 FUSE 挂载，会更容易理解“同一个路径为什么可以换一种文件系统实现”。

例子一：挂载一个 `tmpfs` 内存文件系统

`tmpfs` 是 Linux 提供的内存文件系统，文件内容主要保存在内存中。

```
mkdir -p ~/mount_demo/tmpfs
sudo mount -t tmpfs -o size=32M tmpfs ~/mount_demo/tmpfs
```

现在 `~/mount_demo/tmpfs` 这个目录背后就不是普通磁盘目录，而是一个 32 MB 的内存文件系统：

```
df -h ~/mount_demo/tmpfs
echo "hello tmpfs" > ~/mount_demo/tmpfs/a.txt
cat ~/mount_demo/tmpfs/a.txt
```

卸载：

```
sudo umount ~/mount_demo/tmpfs
```

卸载后，`~/mount_demo/tmpfs` 仍然是一个普通目录，但刚才写到 `tmpfs` 中的文件不会留在原目录里。

例子二：把 `proc` 文件系统挂载到自己的目录

`proc` 是 Linux 内核提供的伪文件系统，里面的内容不是磁盘文件，而是内核和进程信息。

```
mkdir -p ~/mount_demo/proc_view
sudo mount -t proc proc ~/mount_demo/proc_view
```

观察：

```
ls ~/mount_demo/proc_view
cat ~/mount_demo/proc_view/cpuinfo | head
cat ~/mount_demo/proc_view/meminfo | head
```

卸载：

```
sudo umount ~/mount_demo/proc_view
```

这个例子说明：文件系统不一定来自磁盘。`cpuinfo` 和 `meminfo` 不是磁盘上提前保存好的普通文件，而是 `proc` 文件系统根据内核数据结构动态生成的虚拟文件。

这两个例子有一个共同点：**都是把某个文件系统实现接到目录树的某个路径上**。区别在于，`tmpfs` 和 `proc` 由 Linux 内核已有代码实现，而本实验的 `memfs` 将由你自己的用户态程序实现。

3. FUSE 是什么

FUSE 是 Filesystem in Userspace 的缩写，意思是“用户态文件系统”。它提供了一套机制，让普通用户程序也可以实现文件系统逻辑，而不必把代码写进 Linux 内核。

传统文件系统驱动通常运行在内核态，例如 `ext4`、`xfs`。它们直接处理 VFS 发来的请求，再读取或修改磁盘数据。FUSE 的思路不同：它在内核中保留一个很薄的转发层，把真正的文件系统逻辑放到用户态进程中执行。

FUSE 通常由三部分组成：

1. **FUSE 内核模块**：接入 Linux VFS。当某个路径属于 FUSE 挂载点时，它负责接收 VFS 请求。
2. **/dev/fuse 字符设备**：内核和用户态 FUSE 程序之间的通信通道。内核把请求写到这里，用户态程序从这里取请求并返回结果。
3. **用户态 FUSE 程序**：也就是本实验中的 `memfs`。它通过 FUSE 库注册一组回调函数，例如 `getattr`、`readdir`、`read`、`write`，并在用户态处理文件系统请求。

本实验使用的 `libfuse` 是 FUSE 的用户态开发库。你在代码中写：

```
static struct fuse_operations memfs_oper = {
    .getattr = memfs_getattr,
    .readdir = memfs_readdir,
    .read = memfs_read,
    .write = memfs_write,
};

int main(int argc, char *argv[]) {
    return fuse_main(argc, argv, &memfs_oper, NULL);
}
```

这段代码的含义是：把 `memfs_oper` 这张“操作表”交给 FUSE 库。之后，当内核转发来“获取属性”“列目录”“读文件”“写文件”等请求时，FUSE 库会调用你表中对应的函数。

所以，FUSE 不是某一个单独的函数，也不是一个普通 C 库那么简单。它是一套跨越内核态和用户态的文件系统框架：

VFS

- > FUSE 内核模块
- > /dev/fuse
- > libfuse
- > 你写的回调函数

FUSE 的好处是安全、方便调试、开发成本低。你的文件系统代码即使崩溃，通常也只是用户态进程崩溃，而不是让整个内核崩溃。代价是请求需要在内核态和用户态之间来回切换，性能通常不如直接运行在内核中的真实文件系统驱动。

4. 本实验的 FUSE 挂载与调用流程

本实验运行：

```
gcc -Wall memfs.c `pkg-config fuse --cflags --libs` -o memfs
mkdir -p ~/memfs_mount
./memfs ~/memfs_mount
```

其中第一行把 `memfs.c` 编译成可执行文件 `memfs`，第二行创建挂载点目录，第三行启动 FUSE 程序并挂载到 `~/memfs_mount`。

默认情况下，FUSE 通常要求挂载点目录为空。如果是为了观察“挂载后原目录内容被遮住、卸载后重新出现”的效果，可以临时使用：

```
./memfs -o nonempty ~/memfs_mount
```

这个选项适合教学演示；普通使用时仍建议选择空目录作为挂载点。

启动 `./memfs ~/memfs_mount` 这一步不是简单运行一个普通程序。程序中的 `fuse_main` 会和 FUSE 机制配合，向内核注册一个挂载点：

`~/memfs_mount` 这个目录下面的文件系统请求，交给当前这个用户态 `memfs` 程序处理。

之后执行：

```
ls ~/memfs_mount
cat ~/memfs_mount/a.txt
echo "hello" > ~/memfs_mount/a.txt
```

`ls`、`cat`、`echo` 本身没有被修改，也不知道你写了哪些 C 函数。它们仍然只是调用系统调用。变化发生在 VFS：VFS 发现路径位于 FUSE 挂载点下，于是把请求通过 FUSE 转发给你的用户态程序。

本实验的调用流程可以简化为：

```
用户程序
-> open/read/write/unlink 等系统调用
-> Linux VFS
-> 挂载表判断路径属于 FUSE
-> FUSE 内核模块和 /dev/fuse
-> 用户态 memfs 程序
-> memfs_getattr / memfs_readdir / memfs_read / memfs_write 等回调
-> 返回结果和错误码
```

因此，准确地说，你写的程序没有拦截 `ls`，也没有替换系统调用；你只是把一个目录挂载成由自己程序负责处理的文件系统。

4.1 本实验文件系统与真实磁盘文件系统的区别

本实验的 memfs 和真实文件系统有相同的一面：它们都要回答 VFS 的问题。

例如：

- 这个路径是否存在？
- 它是目录还是普通文件？
- 目录下有哪些名字？
- 从某个 offset 开始能读出多少数据？
- 写入后文件大小是否改变？
- 删除文件后目录项是否消失？

但它们的数据来源和完整性能力差别很大：

对比项	本实验 memfs	真实磁盘文件系统
运行位置	用户态进程	通常在内核文件系统驱动中
数据保存位置	C 程序里的数组	磁盘、SSD 或镜像文件中的数据块
元数据结构	简化的 <code>mem_file_t</code> 数组	inode、目录项、位图、日志等结构
持久性	进程退出后数据消失	断电或重启后通常仍然存在
容量管理	固定 <code>MAX_FILES</code> 和 <code>MAX_FILE_SIZE</code>	管理大量块、空闲空间和碎片
崩溃恢复	基础任务不处理	常见文件系统会使用日志或一致性检查
并发控制	基础任务不处理	需要锁、引用计数、缓存一致性等机制

所以，本实验不是在复刻 ext4，而是在实现一个最小模型：用内存数组模拟文件系统需要回答的核心问题，让学生看见 VFS 如何通过挂载点把普通文件操作转交给不同的文件系统实现。

5. 本实验的最小文件系统模型

基础任务只实现根目录下的普通文件。可以把内存中的文件表理解为一个极简 inode 表和目录项表的合并版本。

每个文件至少需要保存：

1. 是否正在使用。
2. 文件名。
3. 文件内容缓冲区。
4. 当前文件大小。

真实文件系统通常会把目录项、inode、数据块、空闲空间管理分开维护。本实验为了突出 FUSE 回调和文件操作语义，将它们合并在一个结构体数组中。

6. 核心回调职责

6.1 getattr

`getattr` 用于回答“这个路径是什么”。它类似 `stat`，需要填充 `struct stat`。

常用字段：

- `st_mode`：文件类型和权限。目录可使用 `S_IFDIR | 0755`，普通文件可使用 `S_IFREG | 0644`。其中 `0755` 和 `0644` 是八进制权限，三位分别表示所有者、同组用户、其他用户；每位中 4 表示读、2 表示写、1 表示执行或进入目录。
- `st_nlink`：链接数。根目录通常可设为 2，普通文件设为 1。
- `st_size`：普通文件当前大小。

如果路径不存在，应返回 `-ENOENT`。如果 `getattr` 的结果错误，VFS 后续可能不会按你预期调用 `open`、`read` 或 `write`。

6.2 readdir

`readdir` 用于回答“目录里有哪些名字”。它会被 `ls` 等操作触发。

根目录至少应填入：

```
filler(buf, ".", NULL, 0);
filler(buf, "..", NULL, 0);
```

然后把当前存在的文件名逐个传给 `filler`。

6.3 create

`create` 用于新建普通文件。许多命令在目标文件不存在时会触发它，例如：

```
echo "hello" > mount/a.txt
```

常见处理逻辑：

1. 解析路径，确认只是在根目录下创建一级文件。
2. 检查是否重名。
3. 找到空闲文件槽位。
4. 初始化文件名、大小和内容缓冲区。

6.4 open

`open` 用于打开已有文件。基础任务中可以只检查文件是否存在。若文件不存在，返回 `-ENOENT`。

如果想进一步限制权限，可以检查 `fi->flags` 中的访问模式，但基础任务不强制要求只读或只写限制。

6.5 read

`read` 的关键是正确处理 `offset` 和 `size`。

如果文件内容为：

```
hello world
```

当 `offset = 6`、`size = 5` 时，应返回：

```
world
```

基本规则：

1. 如果 `offset` 已经大于或等于文件大小，返回 0，表示 EOF。
2. 如果请求范围超过文件末尾，只返回实际存在的部分。
3. 返回值是实际复制到 `buf` 的字节数。

6.6 write

`write` 也必须处理 `offset`。覆盖写和追加写都是通过不同 `offset` 表达的。

例如：

```
echo "abc" > a.txt  
printf "z" >> a.txt
```

追加写通常会以当前文件大小作为 `offset`。写入后，如果 `offset + size` 超过原文件大小，需要更新文件大小。

如果写入超过本实验规定的 `MAX_FILE_SIZE`，应返回错误，例如 `-EFBIG`。

6.7 truncate

`truncate` 用于改变文件长度。很多 shell 重定向和编辑器保存流程都会涉及它。

当文件缩小时，文件大小直接变小即可。被截掉的数据之后不应再被读出。

当文件放大时，新增区域应视为 0。基础任务可以显式用 `memset` 清零新增范围。

6.8 unlink

`unlink` 用于删除普通文件。基础任务中可以把对应槽位的 `used` 标志清除，并清理文件名、大小和内容。删除后，该文件不应再出现在 `readdir` 中，`getattr` 和 `open` 也应返回不存在。

7. 路径解析与错误码

本实验只支持一级路径：

- 合法： `/a.txt`
- 合法： `/note`
- 不作为普通文件处理： `/`
- 不支持： `/dir/a.txt`

建议把路径解析集中放在一个辅助函数中，而不是在每个回调里重复写字符串判断。

常用错误码：

- `-ENOENT`：没有这个文件或目录；在本实验中也可用于“不支持这种路径”，例如 `/dir/a.txt`。
- `-EEXIST`：文件已经存在；例如创建 `/a.txt` 时，内存文件表里已经有 `a.txt`。
- `-ENOSPC`：没有空间；在本实验中通常表示 `files[]` 已经没有空槽位。
- `-ENAMETOOLONG`：文件名太长；例如超过 `MAX_NAME_LEN`。
- `-EFBIG`：文件太大；在本实验中通常表示写入或截断后的大小超过 `MAX_FILE_SIZE`。
- `-EINVAL`：参数非法；例如 `offset` 或 `size` 不合理。
- `-EISDIR`：把目录当成普通文件处理；例如把 `/` 交给只处理普通文件的辅助函数。
- `-ENOTDIR`：路径中间某一层不是目录；例如已经存在普通文件 `/docs`，却访问 `/docs/a.txt`。
- `-ENOTEMPTY`：目录不是空的；例如目录中还有文件或子目录时执行 `rmdir`。
- `-EACCES`：权限不允许；基础任务不一定用到，但限制读写权限时常见。

FUSE 回调通常返回负的 `errno` 值，而不是直接设置 `errno`。

8. 进阶任务实现提示

8.1 挑战一：多级目录内存文件系统

多级目录不能再只用“文件名 -> 内容”的一张表来描述。更合适的做法是把目录和普通文件都看成节点，每个节点至少记录：

1. 节点是否正在使用。
2. 节点类型：目录或普通文件。
3. 节点名：只保存当前层的名字，例如 `week1`，不要保存完整路径 `/docs/week1`。
4. 父节点下标：用于表示这个节点属于哪个目录。
5. 普通文件的内容缓冲区和当前大小。

根目录可以固定占用一个节点，例如下标 0，名字可以设为 `/`，类型为目录，父节点可以设为 -1。这样所有路径都可以从根节点开始逐级查找。

路径解析的关键是把完整路径拆成多个组件。例如：

```
/docs/week1/a.txt
=> docs
=> week1
=> a.txt
```

查找已有路径时，需要从根目录开始，依次在“当前目录的直接子节点”中寻找下一个名字。每找到一层，就进入这一层继续找。中间组件必须是目录；最后一个组件可以是目录，也可以是普通文件，具体取决于当前回调要做什么。

创建新节点时，可以把路径分成“父目录路径”和“新节点名”两部分：

```
/docs/week1/a.txt
父目录路径: /docs/week1
新节点名: a.txt
```

然后先确认父目录存在且确实是目录，再检查父目录下是否已有同名子节点。只有这些条件都满足，才能分配新的节点槽位。

几个回调的实现重点如下：

1. `getattr`：找到目标节点后，根据节点类型填写 `S_IFDIR` 或 `S_IFREG`；普通文件还要填写 `st_size`。
2. `readdir`：找到目标目录节点后，只遍历它的直接子节点；子节点的 `parent` 等于当前目录下标时，才加入 `filler`。
3. `mkdir`：解析父目录和新目录名，在父目录下创建一个目录节点。
4. `rmdir`：目标必须是目录，并且不能存在任何以它为父节点的子节点。
5. `create`：解析父目录和新文件名，在父目录下创建一个普通文件节点。
6. `unlink`：目标必须是普通文件，删除后应释放对应节点。
7. `read`、`write`、`truncate`：目标必须是普通文件；找到节点后，文件内容处理规则可以沿用基础任务。

常见错误码可以按语义选择：路径不存在用 `-ENOENT`；路径中间某一层不是目录用 `-ENOTDIR`；创建时同名节点已存在用 `-EEXIST`；删除非空目录用 `-ENOTEMPTY`；把目录当普通文件操作用 `-EISDIR`。这些错误码的作用是让 shell 命令得到符合直觉的失败原因。

8.2 挑战二：简化透明加密透传文件系统

passthrough 文件系统的核心是：FUSE 挂载点本身不保存最终数据，而是把操作转发到一个后端真实目录。透明加密版本在转发时增加一层内容变换。

建议把挑战范围控制为根目录普通文件，并使用 XOR 加密。XOR 的好处是密文长度和明文长度相同，因此 `offset` 不需要重新映射。写入时，对 `buf` 中的每个字节和密钥做 XOR，再写入后端文件；读取时，从后端文件读取密文字节，再做同样的 XOR 还原明文。

实现时需要特别注意：后端路径必须由固定后端目录和普通文件名拼接得到，不要允许 `../` 这类路径逃出后端目录。`getattr`、`readdir`、`create`、`open`、`read`、`write`、`truncate`、`unlink` 都需要围绕后端文件状态实现。基础挑战不要求支持子目录、文件名加密、AES 或符号链接。